Section A

1. Basic DDL Commands

1. Create a database schema for a college academic management system. The system should manage students, departments, faculty, courses, enrollments, results, and events.

Table Name: DEPARTMENT **Description:** Stores the details of each academic department.

Column Name	Description	Data Type	Constraint
dept_id	Department ID	VARCHAR2(10)	PRIMARY KEY
dept_name	Department Name	VARCHAR2(20)	NOT NULL, UNIQUE
hod_name	Head of Department	VARCHAR2(50)	–
office_location	Office Location	VARCHAR2(50)	–
contact_number	Department Contact Number	VARCHAR2(15)	–

CODE:

```
CREATE TABLE department (
    dept_id         VARCHAR2(10)  PRIMARY KEY,
    dept_name       VARCHAR2(20)  NOT NULL UNIQUE,
    hod_name        VARCHAR2(50),
    office_location VARCHAR2(50),
    contact_number  VARCHAR2(15)
);
```

OUTPUT:

```
Table created.

 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 DEPT_ID                                   NOT NULL VARCHAR2(10)
 DEPT_NAME                                 NOT NULL VARCHAR2(20)
 HOD_NAME                                           VARCHAR2(50)
 OFFICE_LOCATION                                    VARCHAR2(50)
 CONTACT_NUMBER                                     VARCHAR2(15)
```

Table Name: STUDENT **Description:** Stores the personal and academic details of every student.

Column Name	Description	Data Type	Constraint
student_id	Student ID	CHAR(5)	PRIMARY KEY
student_name	Student Name	VARCHAR2(50)	NOT NULL
dob	Date of Birth	DATE	–
gender	Gender	CHAR(1)	CHECK (M or F)
mobile_number	Mobile Number	VARCHAR2(10)	–
email_id	Email ID	VARCHAR2(50)	UNIQUE
dept_id	Department ID	VARCHAR2(10)	FOREIGN KEY REFERENCES department (dept_id)
admission_date	Admission Date	DATE	–
cgpa	CGPA	NUMBER(4,2)	CHECK (0 to 10)

CODE:

```
CREATE TABLE student (
    student_id     CHAR(5)       PRIMARY KEY,
    student_name   VARCHAR2(50)  NOT NULL,
    dob            DATE,
    gender         CHAR(1)       CHECK (gender IN ('M', 'F')),
    mobile_number  VARCHAR2(10),
    email_id       VARCHAR2(50)  UNIQUE,
    dept_id        VARCHAR2(10)  REFERENCES department(dept_id),
    admission_date DATE,
    cgpa           NUMBER(4,2)   CHECK (cgpa BETWEEN 0 AND 10)
);
```

OUTPUT:

```
Table created.

 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 STUDENT_ID                                NOT NULL CHAR(5)
 STUDENT_NAME                              NOT NULL VARCHAR2(50)
 DOB                                                DATE
 GENDER                                             CHAR(1)
 MOBILE_NUMBER                                      VARCHAR2(10)
 EMAIL_ID                                           VARCHAR2(50)
 DEPT_ID                                            VARCHAR2(10)
 ADMISSION_DATE                                     DATE
 CGPA                                               NUMBER(4,2)
```

Table Name: FACULTY **Description:** Stores the details of the teaching faculty.

Column Name	Description	Data Type	Constraint
faculty_id	Faculty ID	CHAR(5)	PRIMARY KEY
faculty_name	Faculty Name	VARCHAR2(50)	NOT NULL
qualification	Qualification	VARCHAR2(30)	–
designation	Designation	VARCHAR2(30)	–
joining_date	Date of Joining	DATE	–
salary	Salary	NUMBER(10,2)	CHECK (salary > 0)
email_id	Email ID	VARCHAR2(50)	UNIQUE
dept_id	Department ID	VARCHAR2(10)	FOREIGN KEY REFERENCES department (dept_id)

CODE:

```
CREATE TABLE faculty (
    faculty_id    CHAR(5)       PRIMARY KEY,
    faculty_name  VARCHAR2(50)  NOT NULL,
    qualification VARCHAR2(30),
    designation   VARCHAR2(30),
    joining_date  DATE,
    salary        NUMBER(10,2)  CHECK (salary > 0),
    email_id      VARCHAR2(50)  UNIQUE,
    dept_id       VARCHAR2(10)  REFERENCES department(dept_id)
);
```

OUTPUT:

```
Table created.

 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 FACULTY_ID                                NOT NULL CHAR(5)
 FACULTY_NAME                              NOT NULL VARCHAR2(50)
 QUALIFICATION                                      VARCHAR2(30)
 DESIGNATION                                        VARCHAR2(30)
 JOINING_DATE                                       DATE
 SALARY                                             NUMBER(10,2)
 EMAIL_ID                                           VARCHAR2(50)
 DEPT_ID                                            VARCHAR2(10)
```

Table Name: COURSE **Description:** Stores the courses offered by each department.

Column Name	Description	Data Type	Constraint
course_id	Course Code	CHAR(5)	PRIMARY KEY
course_name	Course Name	VARCHAR2(50)	NOT NULL
credits	Number of Credits	NUMBER(1)	CHECK (1 to 6)
semester	Semester	NUMBER(1)	CHECK (1 to 8)
dept_id	Department ID	VARCHAR2(10)	FOREIGN KEY REFERENCES department (dept_id)
faculty_id	Faculty ID	CHAR(5)	FOREIGN KEY REFERENCES faculty (faculty_id)

CODE:

```
CREATE TABLE course (
    course_id   CHAR(5)       PRIMARY KEY,
    course_name VARCHAR2(50)  NOT NULL,
    credits     NUMBER(1)     CHECK (credits BETWEEN 1 AND 6),
    semester    NUMBER(1)     CHECK (semester BETWEEN 1 AND 8),
    dept_id     VARCHAR2(10)  REFERENCES department(dept_id),
    faculty_id  CHAR(5)
        CONSTRAINT fk_course_faculty REFERENCES faculty(faculty_id)
);
```

OUTPUT:

```
Table created.

 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 COURSE_ID                                 NOT NULL CHAR(5)
 COURSE_NAME                               NOT NULL VARCHAR2(50)
 CREDITS                                            NUMBER(1)
 SEMESTER                                           NUMBER(1)
 DEPT_ID                                            VARCHAR2(10)
 FACULTY_ID                                         CHAR(5)
```

Table Name: ENROLLMENT **Description:** Records the courses enrolled by each student.

Column Name	Description	Data Type	Constraint
enrollment_id	Enrollment ID	CHAR(5)	PRIMARY KEY
student_id	Student ID	CHAR(5)	FOREIGN KEY REFERENCES student (student_id)
course_id	Course Code	CHAR(5)	FOREIGN KEY REFERENCES course (course_id)
enrollment_date	Enrollment Date	DATE	DEFAULT SYSDATE
academic_year	Academic Year	VARCHAR2(9)	–

CODE:

```
CREATE TABLE enrollment (
    enrollment_id   CHAR(5)      PRIMARY KEY,
    student_id      CHAR(5)      REFERENCES student(student_id),
    course_id       CHAR(5)      REFERENCES course(course_id),
    enrollment_date DATE         DEFAULT SYSDATE,
    academic_year   VARCHAR2(9),
    UNIQUE (student_id, course_id)
);
```

OUTPUT:

```
Table created.

 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ENROLLMENT_ID                             NOT NULL CHAR(5)
 STUDENT_ID                                         CHAR(5)
 COURSE_ID                                          CHAR(5)
 ENROLLMENT_DATE                                    DATE
 ACADEMIC_YEAR                                      VARCHAR2(9)
```

Table Name: RESULT **Description:** Stores the internal, external and total marks of each enrollment.

Column Name	Description	Data Type	Constraint
result_id	Result ID	CHAR(5)	PRIMARY KEY
enrollment_id	Enrollment ID	CHAR(5)	FOREIGN KEY REFERENCES enrollment (enrollment_id)
internal_marks	Internal Marks	NUMBER(5,2)	CHECK (0 to 50)
external_marks	External Marks	NUMBER(5,2)	CHECK (0 to 50)
total_marks	Total Marks	NUMBER(6,2)	–
grade	Grade	CHAR(1)	CHECK (A, B, C, D, F)
result_status	PASS / FAIL	VARCHAR2(10)	CHECK (PASS or FAIL)

CODE:

```
CREATE TABLE result (
    result_id      CHAR(5)       PRIMARY KEY,
    enrollment_id  CHAR(5)       REFERENCES enrollment(enrollment_id),
    internal_marks NUMBER(5,2)   CHECK (internal_marks BETWEEN 0 AND 50),
    external_marks NUMBER(5,2)   CHECK (external_marks BETWEEN 0 AND 50),
    total_marks    NUMBER(6,2),
    grade          CHAR(1)
        CHECK (grade IN ('A', 'B', 'C', 'D', 'F')),
    result_status  VARCHAR2(10)  CHECK (result_status IN ('PASS', 'FAIL'))
);
```

OUTPUT:

```
Table created.

 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 RESULT_ID                                 NOT NULL CHAR(5)
 ENROLLMENT_ID                                      CHAR(5)
 INTERNAL_MARKS                                     NUMBER(5,2)
 EXTERNAL_MARKS                                     NUMBER(5,2)
 TOTAL_MARKS                                        NUMBER(6,2)
 GRADE                                              CHAR(1)
 RESULT_STATUS                                      VARCHAR2(10)
```

Table Name: EVENT **Description:** Stores the details of events conducted by the college.

Column Name	Description	Data Type	Constraint
event_id	Event ID	CHAR(5)	PRIMARY KEY
event_name	Event Name	VARCHAR2(50)	–
event_date	Event Date	DATE	–
venue	Venue	VARCHAR2(50)	–
faculty_id	Coordinator Faculty ID	CHAR(5)	FOREIGN KEY REFERENCES faculty (faculty_id)
max_participants	Maximum Participants	NUMBER(5)	CHECK (max_participants > 0)
registration_fee	Registration Fee	NUMBER(8,2)	CHECK (registration_fee >= 0)

CODE:

```
CREATE TABLE event (
    event_id         CHAR(5)       PRIMARY KEY,
    event_name       VARCHAR2(50),
    event_date       DATE,
    venue            VARCHAR2(50),
    faculty_id       CHAR(5)       REFERENCES faculty(faculty_id),
    max_participants NUMBER(5)     CHECK (max_participants > 0),
    registration_fee NUMBER(8,2)   CHECK (registration_fee >= 0)
);
```

OUTPUT:

```
Table created.

 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 EVENT_ID                                  NOT NULL CHAR(5)
 EVENT_NAME                                         VARCHAR2(50)
 EVENT_DATE                                         DATE
 VENUE                                              VARCHAR2(50)
 FACULTY_ID                                         CHAR(5)
 MAX_PARTICIPANTS                                   NUMBER(5)
 REGISTRATION_FEE                                   NUMBER(8,2)
```

Table Name: EVENT_REGISTRATION **Description:** Records student registrations for each event.

Column Name	Description	Data Type	Constraint
registration_id	Registration ID	CHAR(5)	PRIMARY KEY
event_id	Event ID	CHAR(5)	FOREIGN KEY REFERENCES event (event_id)
student_id	Student ID	CHAR(5)	FOREIGN KEY REFERENCES student (student_id)
registration_date	Registration Date	DATE	DEFAULT SYSDATE
participation_status	Participation Status	VARCHAR2(15)	CHECK (REGISTERED, ATTENDED, ABSENT)

CODE:

```
CREATE TABLE event_registration (
    registration_id      CHAR(5)       PRIMARY KEY,
    event_id             CHAR(5)       REFERENCES event(event_id),
    student_id           CHAR(5)       REFERENCES student(student_id),
    registration_date    DATE          DEFAULT SYSDATE,
    participation_status VARCHAR2(15)
        CHECK (participation_status IN ('REGISTERED', 'ATTENDED', 'ABSENT'))
);
```

OUTPUT:

```
Table created.

 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 REGISTRATION_ID                           NOT NULL CHAR(5)
 EVENT_ID                                           CHAR(5)
 STUDENT_ID                                         CHAR(5)
 REGISTRATION_DATE                                  DATE
 PARTICIPATION_STATUS                               VARCHAR2(15)
```

2. Add a column named address with datatype VARCHAR2(200) to the STUDENT table.

CODE:

```
ALTER TABLE student ADD (address VARCHAR2(200));
DESC student;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 STUDENT_ID                                NOT NULL CHAR(5)
 STUDENT_NAME                              NOT NULL VARCHAR2(50)
 DOB                                                DATE
 GENDER                                             CHAR(1)
 MOBILE_NUMBER                                      VARCHAR2(10)
 EMAIL_ID                                           VARCHAR2(50)
 DEPT_ID                                            VARCHAR2(10)
 ADMISSION_DATE                                     DATE
 CGPA                                               NUMBER(4,2)
 ADDRESS                                            VARCHAR2(200)
```

3. Add two columns, city and state, to the STUDENT table.

CODE:

```
ALTER TABLE student ADD (city VARCHAR2(30), state VARCHAR2(30));
DESC student;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 STUDENT_ID                                NOT NULL CHAR(5)
 STUDENT_NAME                              NOT NULL VARCHAR2(50)
 DOB                                                DATE
 GENDER                                             CHAR(1)
 MOBILE_NUMBER                                      VARCHAR2(10)
 EMAIL_ID                                           VARCHAR2(50)
 DEPT_ID                                            VARCHAR2(10)
 ADMISSION_DATE                                     DATE
 CGPA                                               NUMBER(4,2)
 ADDRESS                                            VARCHAR2(200)
 CITY                                               VARCHAR2(30)
 STATE                                              VARCHAR2(30)
```

4. Add a column named experience_years to the FACULTY table.

CODE:

```
ALTER TABLE faculty ADD (experience_years NUMBER(2));
DESC faculty;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 FACULTY_ID                                NOT NULL CHAR(5)
 FACULTY_NAME                              NOT NULL VARCHAR2(50)
 QUALIFICATION                                      VARCHAR2(30)
 DESIGNATION                                        VARCHAR2(30)
 JOINING_DATE                                       DATE
 SALARY                                             NUMBER(10,2)
 EMAIL_ID                                           VARCHAR2(50)
 DEPT_ID                                            VARCHAR2(10)
 EXPERIENCE_YEARS                                   NUMBER(2)
```

5. Modify the size of the student_name column to VARCHAR2(100).

CODE:

```
ALTER TABLE student MODIFY (student_name VARCHAR2(100));
DESC student;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 STUDENT_ID                                NOT NULL CHAR(5)
 STUDENT_NAME                              NOT NULL VARCHAR2(100)
 DOB                                                DATE
 GENDER                                             CHAR(1)
 MOBILE_NUMBER                                      VARCHAR2(10)
 EMAIL_ID                                           VARCHAR2(50)
 DEPT_ID                                            VARCHAR2(10)
 ADMISSION_DATE                                     DATE
 CGPA                                               NUMBER(4,2)
 ADDRESS                                            VARCHAR2(200)
 CITY                                               VARCHAR2(30)
 STATE                                              VARCHAR2(30)
```

6. Modify the datatype of mobile_number so that it can store 15 characters.

CODE:

```
ALTER TABLE student MODIFY (mobile_number VARCHAR2(15));
DESC student;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 STUDENT_ID                                NOT NULL CHAR(5)
 STUDENT_NAME                              NOT NULL VARCHAR2(100)
 DOB                                                DATE
 GENDER                                             CHAR(1)
 MOBILE_NUMBER                                      VARCHAR2(15)
 EMAIL_ID                                           VARCHAR2(50)
 DEPT_ID                                            VARCHAR2(10)
 ADMISSION_DATE                                     DATE
 CGPA                                               NUMBER(4,2)
 ADDRESS                                            VARCHAR2(200)
 CITY                                               VARCHAR2(30)
 STATE                                              VARCHAR2(30)
```

7. Rename the column mobile_number in the STUDENT table to contact_number.

CODE:

```
ALTER TABLE student RENAME COLUMN mobile_number TO contact_number;
DESC student;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 STUDENT_ID                                NOT NULL CHAR(5)
 STUDENT_NAME                              NOT NULL VARCHAR2(100)
 DOB                                                DATE
 GENDER                                             CHAR(1)
 CONTACT_NUMBER                                     VARCHAR2(15)
 EMAIL_ID                                           VARCHAR2(50)
 DEPT_ID                                            VARCHAR2(10)
 ADMISSION_DATE                                     DATE
 CGPA                                               NUMBER(4,2)
 ADDRESS                                            VARCHAR2(200)
 CITY                                               VARCHAR2(30)
 STATE                                              VARCHAR2(30)
```

8. Rename the column course_name in the COURSE table to course_title.

CODE:

```
ALTER TABLE course RENAME COLUMN course_name TO course_title;
DESC course;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 COURSE_ID                                 NOT NULL CHAR(5)
 COURSE_TITLE                              NOT NULL VARCHAR2(50)
 CREDITS                                            NUMBER(1)
 SEMESTER                                           NUMBER(1)
 DEPT_ID                                            VARCHAR2(10)
 FACULTY_ID                                         CHAR(5)
```

9. Delete the newly added address column from the STUDENT table.

CODE:

```
ALTER TABLE student DROP COLUMN address;
DESC student;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 STUDENT_ID                                NOT NULL CHAR(5)
 STUDENT_NAME                              NOT NULL VARCHAR2(100)
 DOB                                                DATE
 GENDER                                             CHAR(1)
 CONTACT_NUMBER                                     VARCHAR2(15)
 EMAIL_ID                                           VARCHAR2(50)
 DEPT_ID                                            VARCHAR2(10)
 ADMISSION_DATE                                     DATE
 CGPA                                               NUMBER(4,2)
 CITY                                               VARCHAR2(30)
 STATE                                              VARCHAR2(30)
```

10. Drop the qualification column from the FACULTY table.

CODE:

```
ALTER TABLE faculty DROP COLUMN qualification;
DESC faculty;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 FACULTY_ID                                NOT NULL CHAR(5)
 FACULTY_NAME                              NOT NULL VARCHAR2(50)
 DESIGNATION                                        VARCHAR2(30)
 JOINING_DATE                                       DATE
 SALARY                                             NUMBER(10,2)
 EMAIL_ID                                           VARCHAR2(50)
 DEPT_ID                                            VARCHAR2(10)
 EXPERIENCE_YEARS                                   NUMBER(2)
```

11. Add a CHECK constraint to ensure that faculty salary is greater than zero.

CODE:

```
ALTER TABLE faculty ADD CONSTRAINT chk_faculty_salary CHECK (salary > 0);
UPDATE faculty SET salary = -5000 WHERE faculty_id = 'F101';
```

OUTPUT:

```
Table altered.

UPDATE faculty SET salary = -5000 WHERE faculty_id = 'F101'
*
ERROR at line 1:
ORA-02290: check constraint (MCA.SYS_C009682) violated
```

12. Add a CHECK constraint to ensure that student CGPA remains between 0 and 10.

CODE:

```
ALTER TABLE student
    ADD CONSTRAINT chk_student_cgpa CHECK (cgpa BETWEEN 0 AND 10);
UPDATE student SET cgpa = 12 WHERE student_id = 'S101';
```

OUTPUT:

```
Table altered.

UPDATE student SET cgpa = 12 WHERE student_id = 'S101'
*
ERROR at line 1:
ORA-02290: check constraint (MCA.SYS_C009677) violated
```

13. Add a NOT NULL constraint to the event_name column.

CODE:

```
ALTER TABLE event MODIFY (event_name NOT NULL);
DESC event;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 EVENT_ID                                  NOT NULL CHAR(5)
 EVENT_NAME                                NOT NULL VARCHAR2(50)
 EVENT_DATE                                         DATE
 VENUE                                              VARCHAR2(50)
 FACULTY_ID                                         CHAR(5)
 MAX_PARTICIPANTS                                   NUMBER(5)
 REGISTRATION_FEE                                   NUMBER(8,2)
```

14. Add a unique constraint to the contact_number column of the STUDENT table.

CODE:

```
ALTER TABLE student
    ADD CONSTRAINT uq_student_contact UNIQUE (contact_number);
DESC student;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 STUDENT_ID                                NOT NULL CHAR(5)
 STUDENT_NAME                              NOT NULL VARCHAR2(100)
 DOB                                                DATE
 GENDER                                             CHAR(1)
 CONTACT_NUMBER                                     VARCHAR2(15)
 EMAIL_ID                                           VARCHAR2(50)
 DEPT_ID                                            VARCHAR2(10)
 ADMISSION_DATE                                     DATE
 CGPA                                               NUMBER(4,2)
 CITY                                               VARCHAR2(30)
 STATE                                              VARCHAR2(30)
```

15. Rename the EVENT table to COLLEGE_EVENT.

CODE:

```
ALTER TABLE event RENAME TO college_event;
DESC college_event;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 EVENT_ID                                  NOT NULL CHAR(5)
 EVENT_NAME                                NOT NULL VARCHAR2(50)
 EVENT_DATE                                         DATE
 VENUE                                              VARCHAR2(50)
 FACULTY_ID                                         CHAR(5)
 MAX_PARTICIPANTS                                   NUMBER(5)
 REGISTRATION_FEE                                   NUMBER(8,2)
```

16. Rename the COLLEGE_EVENT table back to EVENT.

CODE:

```
ALTER TABLE college_event RENAME TO event;
DESC event;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 EVENT_ID                                  NOT NULL CHAR(5)
 EVENT_NAME                                NOT NULL VARCHAR2(50)
 EVENT_DATE                                         DATE
 VENUE                                              VARCHAR2(50)
 FACULTY_ID                                         CHAR(5)
 MAX_PARTICIPANTS                                   NUMBER(5)
 REGISTRATION_FEE                                   NUMBER(8,2)
```

17. Add a column named hod_faculty_id to the DEPARTMENT table.

CODE:

```
ALTER TABLE department ADD (hod_faculty_id CHAR(5));
DESC department;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 DEPT_ID                                   NOT NULL VARCHAR2(10)
 DEPT_NAME                                 NOT NULL VARCHAR2(20)
 HOD_NAME                                           VARCHAR2(50)
 OFFICE_LOCATION                                    VARCHAR2(50)
 CONTACT_NUMBER                                     VARCHAR2(15)
 HOD_FACULTY_ID                                     CHAR(5)
```

18. Create a foreign-key constraint linking hod_faculty_id in DEPARTMENT with faculty_id in FACULTY.

CODE:

```
ALTER TABLE department
    ADD CONSTRAINT fk_dept_hod FOREIGN KEY (hod_faculty_id)
    REFERENCES faculty(faculty_id);
DESC department;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 DEPT_ID                                   NOT NULL VARCHAR2(10)
 DEPT_NAME                                 NOT NULL VARCHAR2(20)
 HOD_NAME                                           VARCHAR2(50)
 OFFICE_LOCATION                                    VARCHAR2(50)
 CONTACT_NUMBER                                     VARCHAR2(15)
 HOD_FACULTY_ID                                     CHAR(5)
```

19. Remove the existing hod_name column from the DEPARTMENT table, as required in the database redesign, and display the structure of the modified table.

CODE:

```
ALTER TABLE department DROP COLUMN hod_name;
DESC department;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 DEPT_ID                                   NOT NULL VARCHAR2(10)
 DEPT_NAME                                 NOT NULL VARCHAR2(20)
 OFFICE_LOCATION                                    VARCHAR2(50)
 CONTACT_NUMBER                                     VARCHAR2(15)
 HOD_FACULTY_ID                                     CHAR(5)
```

20. Disable the foreign-key constraint connecting COURSE and FACULTY.

CODE:

```
ALTER TABLE course DISABLE CONSTRAINT fk_course_faculty;
DESC course;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 COURSE_ID                                 NOT NULL CHAR(5)
 COURSE_TITLE                              NOT NULL VARCHAR2(50)
 CREDITS                                            NUMBER(1)
 SEMESTER                                           NUMBER(1)
 DEPT_ID                                            VARCHAR2(10)
 FACULTY_ID                                         CHAR(5)
```

21. Enable the previously disabled foreign-key constraint.

CODE:

```
ALTER TABLE course ENABLE CONSTRAINT fk_course_faculty;
DESC course;
```

OUTPUT:

```
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 COURSE_ID                                 NOT NULL CHAR(5)
 COURSE_TITLE                              NOT NULL VARCHAR2(50)
 CREDITS                                            NUMBER(1)
 SEMESTER                                           NUMBER(1)
 DEPT_ID                                            VARCHAR2(10)
 FACULTY_ID                                         CHAR(5)
```

2. Basic DML Commands

1. Insert a new department with Department ID D01, Department Name Computer Science, Office Location Carmel Block and Contact Number 9876543210.

CODE:

```
INSERT INTO department (dept_id, dept_name, office_location, contact_number)
VALUES ('D01', 'Computer Science', 'Carmel Block', '9876543210');

SELECT * FROM department;
```

OUTPUT:

```
DEPT_ID  DEPT_NAME         OFFICE_LOCATION  CONTACT_NUMBER  HOD_FACULTY_ID
-------  -----------------  ---------------  --------------  --------------
D01      Computer Science  Carmel Block     9876543210

1 row selected.
```

2. Insert the student record of Akhil Raj (S101) into the STUDENT table.

CODE:

```
INSERT INTO student (student_id, student_name, dob, gender, contact_number,
                     email_id, dept_id, admission_date, cgpa)
VALUES ('S101', 'Akhil Raj', TO_DATE('15-08-2005', 'DD-MM-YYYY'), 'M',
        '9876501234', 'akhil@gmail.com', 'D01', SYSDATE, 8.2);

SELECT * FROM student;
```

OUTPUT:

```
STUDENT_ID STUDENT_NAME DOB       GENDER CONTACT_NUMBER EMAIL_ID        DEPT_ID ADMISSION_DATE CGPA CITY STATE
---------- ------------ --------- ------ -------------- ---------------- ------- -------------- ---- ---- -----
S101       Akhil Raj    15-AUG-05 M      9876501234     akhil@gmail.com D01     24-OCT-26       8.2

1 row selected.
```

3. Insert a student record without providing the CGPA value.

CODE:

```
INSERT INTO student (student_id, student_name, dob, gender, contact_number,
                     email_id, dept_id, admission_date)
VALUES ('S102', 'Neha Thomas', TO_DATE('22-03-2006', 'DD-MM-YYYY'), 'F',
        '9876501235', 'neha@gmail.com', 'D01', SYSDATE);

SELECT * FROM student;
```

OUTPUT:

```
STUDENT_ID STUDENT_NAME DOB       GENDER CONTACT_NUMBER EMAIL_ID        DEPT_ID ADMISSION_DATE CGPA CITY STATE
---------- ------------ --------- ------ -------------- --------------- ------- -------------- ---- ---- -----
S101       Akhil Raj    15-AUG-05 M      9876501234     akhil@gmail.com D01     24-OCT-26       8.2
S102       Neha Thomas  22-MAR-06 F      9876501235     neha@gmail.com  D01     24-OCT-26

2 rows selected.
```

4. Insert two faculty records into the FACULTY table using a single INSERT ALL statement.

CODE:

```
INSERT ALL
  INTO faculty (faculty_id, faculty_name, designation, joining_date, salary,
                email_id, dept_id)
  VALUES ('F102', 'Ravi Kumar', 'Assistant Professor', TO_DATE('10-07-2024','DD-MM-YYYY'),
          25000, 'ravi@rajagiri.edu', 'D01')
  INTO faculty (faculty_id, faculty_name, designation, joining_date, salary,
                email_id, dept_id)
  VALUES ('F103', 'Anita Thomas', 'Professor', TO_DATE('01-06-2011','DD-MM-YYYY'),
          90000, 'anita@rajagiri.edu', 'D01')
SELECT * FROM dual;

SELECT * FROM faculty;
```

OUTPUT:

```
FACULTY_ID  FACULTY_NAME  DESIGNATION          JOINING_DATE  SALARY  EMAIL_ID            DEPT_ID  EXPERIENCE_YEARS
----------  ------------  -------------------  ------------  ------  ------------------  -------  ----------------
F102        Ravi Kumar    Assistant Professor  10-JUL-24      25000  ravi@rajagiri.edu   D01
F103        Anita Thomas  Professor            01-JUN-11      90000  anita@rajagiri.edu  D01

2 rows selected.
```

5. Insert the faculty member Meera Joseph (F101) into the FACULTY table.

CODE:

```
INSERT INTO faculty (faculty_id, faculty_name, designation, joining_date, salary,
                     email_id, dept_id)
VALUES ('F101', 'Meera Joseph', 'Assistant Professor', TO_DATE('01-06-2022','DD-MM-YYYY'),
        55000, 'meera@rajagiri.edu', 'D01');

SELECT * FROM faculty;
```

OUTPUT:

```
FACULTY_ID  FACULTY_NAME  DESIGNATION          JOINING_DATE  SALARY  EMAIL_ID            DEPT_ID  EXPERIENCE_YEARS
----------  ------------  -------------------  ------------  ------  ------------------  -------  ----------------
F102        Ravi Kumar    Assistant Professor  10-JUL-24      25000  ravi@rajagiri.edu   D01
F103        Anita Thomas  Professor            01-JUN-11      90000  anita@rajagiri.edu  D01
F101        Meera Joseph  Assistant Professor  01-JUN-22      55000  meera@rajagiri.edu  D01

3 rows selected.
```

6. Insert a course titled Database Management Systems with four credits for Semester 3.

CODE:

```
INSERT INTO course (course_id, course_title, credits, semester, dept_id, faculty_id)
VALUES ('C101', 'Database Management Systems', 4, 3, 'D01', 'F101');

SELECT * FROM course;
```

OUTPUT:

```
COURSE_ID  COURSE_TITLE                 CREDITS  SEMESTER  DEPT_ID  FACULTY_ID
---------  ---------------------------  -------  --------  -------  ----------
C101       Database Management Systems        4         3  D01      F101

1 row selected.
```

7. Insert a course without mentioning the column names. Ensure that the values are entered in the correct table-column order.

CODE:

```
INSERT INTO course
VALUES ('C102', 'Operating Systems', 3, 8, 'D01', 'F102');

SELECT * FROM course;
```

OUTPUT:

```
COURSE_ID  COURSE_TITLE                 CREDITS  SEMESTER  DEPT_ID  FACULTY_ID
---------  --------------------------  -------  --------  -------  ----------
C101       Database Management Systems        4         3  D01      F101
C102       Operating Systems                  3         8  D01      F102

2 rows selected.
```

8. Insert an enrollment record and allow the enrollment_date column to take its default value.

CODE:

```
INSERT INTO enrollment (enrollment_id, student_id, course_id, academic_year)
VALUES ('E101', 'S101', 'C101', '2025-2026');

SELECT * FROM enrollment;
```

OUTPUT:

```
ENROLLMENT_ID  STUDENT_ID  COURSE_ID  ENROLLMENT_DATE  ACADEMIC_YEAR
-------------  ----------  ---------  ---------------  -------------
E101           S101        C101       24-OCT-26        2025-2026

1 row selected.
```

9. Insert a result record where the total marks are calculated by adding the internal and external marks.

CODE:

```
INSERT INTO result (result_id, enrollment_id, internal_marks, external_marks,
                    total_marks, grade, result_status)
VALUES ('R101', 'E101', 18, 22, 18 + 22, 'F', 'FAIL');

SELECT * FROM result;
```

OUTPUT:

```
RESULT_ID  ENROLLMENT_ID  INTERNAL_MARKS  EXTERNAL_MARKS  TOTAL_MARKS  GRADE  RESULT_STATUS
---------  -------------  --------------  --------------  -----------  -----  -------------
R101       E101                       18              22           40  F      FAIL

1 row selected.
```

10. Insert three events into the EVENT table using an Oracle INSERT ALL statement.

CODE:

```
INSERT ALL
  INTO event VALUES ('EV01', 'Tech Fest', TO_DATE('15-03-2026','DD-MM-YYYY'),
                     'Main Auditorium', 'F101', 150, 200)
  INTO event VALUES ('EV02', 'Workshop on AI', TO_DATE('10-04-2024','DD-MM-YYYY'),
                     'Seminar Hall', 'F102', 50, NULL)
  INTO event VALUES ('EV03', 'National Seminar', TO_DATE('20-11-2025','DD-MM-YYYY'),
                     'Seminar Hall', 'F103', 300, 100)
SELECT * FROM dual;

SELECT * FROM event;
```

OUTPUT:

```
EVENT_ID  EVENT_NAME        EVENT_DATE  VENUE            FACULTY_ID  MAX_PARTICIPANTS  REGISTRATION_FEE
--------  ----------------  ----------  ---------------  ----------  ----------------  ----------------
EV01      Tech Fest         15-MAR-26   Main Auditorium  F101                   150               200
EV02      Workshop on AI    10-APR-24   Seminar Hall     F102                    50
EV03      National Seminar  20-NOV-25   Seminar Hall     F103                   300               100

3 rows selected.
```

11. Insert an event-registration record and allow the registration date to take the current date automatically.

CODE:

```
INSERT INTO event_registration (registration_id, event_id, student_id,
                                participation_status)
VALUES ('REG01', 'EV01', 'S101', 'ABSENT');

SELECT * FROM event_registration;
```

OUTPUT:

```
REGISTRATION_ID  EVENT_ID  STUDENT_ID  REGISTRATION_DATE  PARTICIPATION_STATUS
---------------  --------  ----------  -----------------  --------------------
REG01            EV01      S101        24-OCT-26          ABSENT

1 row selected.
```

12. Update the CGPA of student S101 to 8.7.

CODE:

```
UPDATE student SET cgpa = 8.7 WHERE student_id = 'S101';
```

OUTPUT:

```
1 row updated.
Commit complete.
```

13. Increase the salary of all faculty members by 10%.

CODE:

```
UPDATE faculty SET salary = salary * 1.10;
```

OUTPUT:

```
3 rows updated.
Commit complete.
```

14. Change the office location of the Computer Science department to Golden Aureole Block.

CODE:

```
UPDATE department SET office_location = 'Golden Aureole Block'
WHERE dept_name = 'Computer Science';
```

OUTPUT:

```
1 row updated.
Commit complete.
```

15. Update the credits of the course Database Management Systems from 4 to 5.

CODE:

```
UPDATE course SET credits = 5
WHERE course_title = 'Database Management Systems';
```

OUTPUT:

```
1 row updated.
Commit complete.
```

16. Set the result status to PASS for results with total marks greater than or equal to 50, and FAIL for the remaining records.

CODE:

```
UPDATE result
SET result_status = CASE WHEN total_marks >= 50 THEN 'PASS' ELSE 'FAIL' END;
```

OUTPUT:

```
1 row updated.
Commit complete.
```

17. Delete courses offered in Semester 8.

CODE:

```
DELETE FROM course WHERE semester = 8;
```

OUTPUT:

```
1 row deleted.
Commit complete.
```

18. Delete all results with the result status FAIL.

CODE:

```
DELETE FROM result WHERE result_status = 'FAIL';
```

OUTPUT:

```
1 row deleted.
Commit complete.
```

19. Delete event registrations where the participation status is ABSENT.

CODE:

```
DELETE FROM event_registration WHERE participation_status = 'ABSENT';
```

OUTPUT:

```
1 row deleted.
Commit complete.
```

20. Delete all events for which no registration fee is charged.

CODE:

```
DELETE FROM event WHERE registration_fee IS NULL OR registration_fee = 0;
```

OUTPUT:

```
1 row deleted.
Commit complete.
```

3. Basic SQL

1. Display the different faculty designations without duplicates.

CODE:

```
SELECT DISTINCT designation FROM faculty;
```

OUTPUT:

```
DESIGNATION
-------------------
Assistant Professor
Professor
Associate Professor

3 rows selected.
```

2. Display the different semesters in which courses are offered.

CODE:

```
SELECT DISTINCT semester FROM course;
```

OUTPUT:

```
SEMESTER
--------
       3
       1
       5
       7

4 rows selected.
```

3. Display the student name and CGPA using the aliases Student_Name and Grade_Point_Average.

CODE:

```
SELECT student_name AS Student_Name, cgpa AS Grade_Point_Average
FROM student;
```

OUTPUT:

```
STUDENT_NAME    GRADE_POINT_AVERAGE
--------------  -------------------
Akhil Raj       8.7
Neha Thomas
Anjali Menon    9.1
Arjun Nair      8.4
Aparna Pillai   7.2
Rahul Varma     6.5
Sneha George    8.9
Aditya Kumar    5.8
Meera Krishnan  9.6
Anoop Joseph    7.8
Divya Ramesh    8
Kiran Mathew    6.9

12 rows selected.
```

4. Display the faculty name, monthly salary, and annual salary. Calculate annual salary as monthly salary multiplied by 12.

CODE:

```
SELECT faculty_name, salary, salary * 12 AS annual_salary
FROM faculty;
```

OUTPUT:

```
FACULTY_NAME  SALARY  ANNUAL_SALARY
------------  ------  -------------
Ravi Kumar     27500         330000
Anita Thomas   99000        1188000
Meera Joseph   60500         726000
Suresh Menon   95000        1140000
Latha Nair     72000         864000
Vinod Kumar    48000         576000
Priya Sharma   52000         624000
arun das       45000         540000

8 rows selected.
```

5. Display the faculty name, current salary, and salary after a 10% increase without modifying the table.

CODE:

```
SELECT faculty_name, salary, salary * 1.10 AS increased_salary
FROM faculty;
```

OUTPUT:

```
FACULTY_NAME  SALARY  INCREASED_SALARY
------------  ------  ----------------
Ravi Kumar     27500             30250
Anita Thomas   99000            108900
Meera Joseph   60500             66550
Suresh Menon   95000            104500
Latha Nair     72000             79200
Vinod Kumar    48000             52800
Priya Sharma   52000             57200
arun das       45000             49500

8 rows selected.
```

6. Display the result ID, internal marks, external marks, and a calculated total obtained by adding the two marks.

CODE:

```
SELECT result_id, internal_marks, external_marks,
       internal_marks + external_marks AS total
FROM result;
```

OUTPUT:

```
RESULT_ID  INTERNAL_MARKS  EXTERNAL_MARKS  TOTAL
---------  --------------  --------------  -----
R102                   45              40     85
R103                   38              35     73
R104                   42              38     80
R105                   25              20     45
R106                   30              28     58
R107                   48              45     93
R108                   35              30     65
R109                   20              15     35
R110                   40              42     82
R111                   28              25     53
R112                   44              41     85
R113                   33              32     65
R114                   22              18     40
R115                   46              44     90

14 rows selected.
```

7. Display the event name, registration fee, and registration fee after adding 18% tax.

CODE:

```
SELECT event_name, registration_fee,
       registration_fee * 1.18 AS fee_with_tax
FROM event;
```

OUTPUT:

```
EVENT_NAME        REGISTRATION_FEE  FEE_WITH_TAX
----------------  ----------------  ------------
Tech Fest                      200           236
National Seminar               100           118
Cultural Fest                  750           885
Sports Meet                      0             0
Alumni Meet                   1500          1770

5 rows selected.
```

8. Display faculty members whose salary is greater than 50,000.

CODE:

```
SELECT * FROM faculty WHERE salary > 50000;
```

OUTPUT:

```
FACULTY_ID  FACULTY_NAME  DESIGNATION          JOINING_DATE  SALARY  EMAIL_ID             DEPT_ID  EXPERIENCE_YEARS
----------  ------------  -------------------  ------------  ------  -------------------  -------  ----------------
F103        Anita Thomas  Professor            01-JUN-11      99000  anita@rajagiri.edu   D01
F101        Meera Joseph  Assistant Professor  01-JUN-22      60500  meera@rajagiri.edu   D01
F104        Suresh Menon  Professor            12-JUL-08      95000  suresh@rajagiri.edu  D02                    18
F105        Latha Nair    Associate Professor  05-AUG-15      72000  latha@rajagiri.edu   D03                    11
F107        Priya Sharma  Assistant Professor  02-MAR-21      52000  priya@rajagiri.edu   D05                     5

5 rows selected.
```

9. Display courses offered in Semester 3.

CODE:

```
SELECT * FROM course WHERE semester = 3;
```

OUTPUT:

```
COURSE_ID  COURSE_TITLE                CREDITS  SEMESTER  DEPT_ID  FACULTY_ID
---------  --------------------------  -------  --------  -------  ----------
C101       Database Management Systems        5         3  D01      F101
C103       Data Structures                    4         3  D01      F106
C105       Thermodynamics                     4         3  D03      F105

3 rows selected.
```

10. Display results in which total marks are greater than or equal to 80.

CODE:

```
SELECT * FROM result WHERE total_marks >= 80;
```

OUTPUT:

```
RESULT_ID  ENROLLMENT_ID  INTERNAL_MARKS  EXTERNAL_MARKS  TOTAL_MARKS  GRADE  RESULT_STATUS
---------  -------------  --------------  --------------  -----------  -----  -------------
R102       E102                       45              40           85  A      PASS
R104       E104                       42              38           80  A      PASS
R107       E107                       48              45           93  A      PASS
R110       E110                       40              42           82  A      PASS
R112       E112                       44              41           85  A      PASS
R115       E115                       46              44           90  A      PASS

6 rows selected.
```

11. Display events for which the registration fee is greater than 500.

CODE:

```
SELECT * FROM event WHERE registration_fee > 500;
```

OUTPUT:

```
EVENT_ID  EVENT_NAME     EVENT_DATE  VENUE            FACULTY_ID  MAX_PARTICIPANTS  REGISTRATION_FEE
--------  -------------  ----------  ---------------  ----------  ----------------  ----------------
EV04      Cultural Fest  10-FEB-26   Open Ground      F105                     500               750
EV06      Alumni Meet    05-DEC-25   Main Auditorium  F104                      80              1500

2 rows selected.
```

12. Display students whose CGPA is between 7 and 9.

CODE:

```
SELECT student_id, student_name, cgpa, dept_id
FROM student WHERE cgpa BETWEEN 7 AND 9;
```

OUTPUT:

```
STUDENT_ID  STUDENT_NAME   CGPA  DEPT_ID
----------  -------------  ----  -------
S101        Akhil Raj       8.7  D01
S104        Arjun Nair      8.4  D02
S105        Aparna Pillai   7.2  D01
S107        Sneha George    8.9  D02
S110        Anoop Joseph    7.8  D05
S111        Divya Ramesh      8  D03

6 rows selected.
```

13. Display courses offered in Semesters 1, 3, or 5 using the IN operator.

CODE:

```
SELECT course_id, course_title, semester
FROM course WHERE semester IN (1, 3, 5);
```

OUTPUT:

```
COURSE_ID  COURSE_TITLE                 SEMESTER
---------  --------------------------  --------
C101       Database Management Systems         3
C103       Data Structures                     3
C104       Digital Electronics                 1
C105       Thermodynamics                      3
C106       Surveying                           5
C107       Operating Systems                   5
C109       Software Engineering                5
C110       Web Technologies                    1

8 rows selected.
```

14. Display students whose names begin with the letter A.

CODE:

```
SELECT student_id, student_name, cgpa
FROM student WHERE student_name LIKE 'A%';
```

OUTPUT:

```
STUDENT_ID  STUDENT_NAME   CGPA
----------  -------------  ----
S101        Akhil Raj       8.7
S103        Anjali Menon    9.1
S104        Arjun Nair      8.4
S105        Aparna Pillai   7.2
S108        Aditya Kumar    5.8
S110        Anoop Joseph    7.8

6 rows selected.
```

15. Display faculty members whose email addresses belong to the rajagiri.edu domain.

CODE:

```
SELECT faculty_id, faculty_name, email_id
FROM faculty WHERE email_id LIKE '%@rajagiri.edu';
```

OUTPUT:

```
FACULTY_ID  FACULTY_NAME  EMAIL_ID
----------  ------------  --------------------
F102        Ravi Kumar    ravi@rajagiri.edu
F103        Anita Thomas  anita@rajagiri.edu
F101        Meera Joseph  meera@rajagiri.edu
F104        Suresh Menon  suresh@rajagiri.edu
F105        Latha Nair    latha@rajagiri.edu
F107        Priya Sharma  priya@rajagiri.edu

6 rows selected.
```

16. Display students whose CGPA has not been entered.

CODE:

```
SELECT student_id, student_name
FROM student WHERE cgpa IS NULL;
```

OUTPUT:

```
STUDENT_ID  STUDENT_NAME
----------  ------------
S102        Neha Thomas

1 row selected.
```

17. Display all student names in uppercase.

CODE:

```
SELECT UPPER(student_name) AS student_name FROM student;
```

OUTPUT:

```
STUDENT_NAME
--------------
AKHIL RAJ
NEHA THOMAS
ANJALI MENON
ARJUN NAIR
APARNA PILLAI
RAHUL VARMA
SNEHA GEORGE
ADITYA KUMAR
MEERA KRISHNAN
ANOOP JOSEPH
DIVYA RAMESH
KIRAN MATHEW

12 rows selected.
```

18. Display faculty names with the first letter of each word capitalised using INITCAP.

CODE:

```
SELECT INITCAP(faculty_name) AS faculty_name FROM faculty;
```

OUTPUT:

```
FACULTY_NAME
------------
Ravi Kumar
Anita Thomas
Meera Joseph
Suresh Menon
Latha Nair
Vinod Kumar
Priya Sharma
Arun Das

8 rows selected.
```

19. Display student names along with the number of characters in each name.

CODE:

```
SELECT student_name, LENGTH(student_name) AS name_length
FROM student;
```

OUTPUT:

```
STUDENT_NAME    NAME_LENGTH
--------------  -----------
Akhil Raj                 9
Neha Thomas              11
Anjali Menon             12
Arjun Nair               10
Aparna Pillai            13
Rahul Varma              11
Sneha George             12
Aditya Kumar             12
Meera Krishnan           14
Anoop Joseph             12
Divya Ramesh             12
Kiran Mathew             12

12 rows selected.
```

20. Find the total number of students, and the highest, lowest and average CGPA of all students.

CODE:

```
SELECT COUNT(*) AS total_students FROM student;

SELECT MAX(cgpa) AS highest, MIN(cgpa) AS lowest,
       AVG(cgpa) AS average
FROM student;
```

OUTPUT:

```
TOTAL_STUDENTS
--------------
            12

1 row selected.

HIGHEST  LOWEST  AVERAGE
-------  ------  -------
    9.6     5.8      7.9

1 row selected.
```

Section B

4. Advanced SQL

4.1 Select Queries

1. Display the member ID, full name, member type and city of all ACTIVE members, arranged first by member type and then alphabetically by name.

CODE:

```
SELECT member_id, full_name, member_type, city
FROM member
WHERE status = 'ACTIVE'
ORDER BY member_type, full_name;
```

OUTPUT:

```
MEMBER_ID  FULL_NAME      MEMBER_TYPE  CITY
---------  -------------  -----------  --------------
     1003  Meera Thomas   FACULTY      Kakkanad
     1013  Neha Varma     FACULTY      Kakkanad
     1007  Sara Mathew    FACULTY      Kochi
     1004  Rohan Joseph   PUBLIC       Kochi
     1010  Adil Khan      RESEARCHER   Kakkanad
     1005  Ananya Pillai  RESEARCHER   Thrippunithura
     1015  Fathima Noor   RESEARCHER   Aluva
     1001  Aarav Menon    STUDENT      Kochi
     1002  Diya Nair      STUDENT      Aluva
     1009  Ishita Rao     STUDENT      Kochi
     1011  Liya George    STUDENT      Thrippunithura

11 rows selected.
```

2. List the members whose outstanding amount is between 50 and 300, inclusive. Show member ID, name, outstanding amount and status in descending order of outstanding amount.

CODE:

```
SELECT member_id, full_name, outstanding_amt, status
FROM member
WHERE outstanding_amt BETWEEN 50 AND 300
ORDER BY outstanding_amt DESC;
```

OUTPUT:

```
MEMBER_ID  FULL_NAME     OUTSTANDING_AMT  STATUS
---------  ------------  ---------------  -------
     1010  Adil Khan                 250  ACTIVE
     1004  Rohan Joseph              120  ACTIVE
     1006  Nikhil Raj                 75  EXPIRED
     1014  Joel Antony                60  EXPIRED

4 rows selected.
```

3. Display all members who are either STUDENT or RESEARCHER and whose maximum borrowing limit is at least 4 books. Sort by max_books descending.

CODE:

```
SELECT member_id, full_name, member_type, max_books
FROM member
WHERE member_type IN ('RESEARCHER', 'STUDENT')
  AND max_books >= 4
ORDER BY max_books DESC;
```

OUTPUT:

```
MEMBER_ID  FULL_NAME      MEMBER_TYPE  MAX_BOOKS
---------  -------------  -----------  ---------
     1015  Fathima Noor   RESEARCHER          10
     1005  Ananya Pillai  RESEARCHER          10
     1010  Adil Khan      RESEARCHER          10
     1011  Liya George    STUDENT              4
     1014  Joel Antony    STUDENT              4
     1006  Nikhil Raj     STUDENT              4
     1002  Diya Nair      STUDENT              4
     1001  Aarav Menon    STUDENT              4
     1009  Ishita Rao     STUDENT              4

9 rows selected.
```

4. Display member name, join date and the number of complete years they have been members as of SYSDATE. Show the longest-standing members first.

CODE:

```
SELECT full_name, join_date,
       TRUNC(MONTHS_BETWEEN(SYSDATE, join_date) / 12) AS completed_years
FROM member
ORDER BY completed_years DESC;
```

OUTPUT:

```
FULL_NAME        JOIN_DATE  COMPLETED_YEARS
---------------  ---------  ---------------
Adil Khan        01-OCT-19                7
Sara Mathew      10-FEB-20                6
Ananya Pillai    20-AUG-21                5
Meera Thomas     15-JAN-22                4
Joel Antony      15-JUN-22                4
Rohan Joseph     11-SEP-23                3
Nikhil Raj       05-JUN-23                3
Harish Kumar     20-NOV-22                3
Neha Varma       03-MAR-23                3
Ishita Rao       10-JUN-24                2
Fathima Noor     18-APR-24                2
Aarav Menon      10-JUN-24                2
Diya Nair        01-JUL-24                2
Vivek Krishnan   05-JAN-25                1
Liya George      12-JUN-25                1

15 rows selected.
```

5. Display the names and email addresses of members whose name contains the letter ’a’ anywhere, ignoring case, and whose account is ACTIVE.

CODE:

```
SELECT LOWER(full_name) AS name, email
FROM member
WHERE LOWER(full_name) LIKE '%a%'
  AND status = 'ACTIVE';
```

OUTPUT:

```
NAME           EMAIL
-------------  --------------------
aarav menon    aarav.m@example.com
diya nair      diya.n@example.com
meera thomas   meera.t@example.com
rohan joseph   rohan.j@example.com
ananya pillai  ananya.p@example.com
sara mathew    sara.m@example.com
ishita rao     ishita.r@example.com
adil khan      adil.k@example.com
liya george    liya.g@example.com
neha varma     neha.v@example.com
fathima noor   fathima.n@example.com

11 rows selected.
```

6. Produce a member-status summary showing each status, the number of members in that status and the average outstanding amount for that status, rounded to two decimal places.

CODE:

```
SELECT status, COUNT(*) AS no_of_members,
       ROUND(AVG(outstanding_amt), 2) AS average_out_amt
FROM member
GROUP BY status;
```

OUTPUT:

```
STATUS     NO_OF_MEMBERS  AVERAGE_OUT_AMT
---------  -------------  ---------------
ACTIVE                11            39.55
EXPIRED                3               45
SUSPENDED              1              480

3 rows selected.
```

7. Display cities having at least 2 registered members. Show the city, number of members and total outstanding amount for that city, arranged by number of members descending.

CODE:

```
SELECT city, COUNT(*) AS no_of_members,
       SUM(outstanding_amt) AS total_out_amt
FROM member
GROUP BY city
HAVING COUNT(member_id) >= 2
ORDER BY no_of_members DESC;
```

OUTPUT:

```
CITY            NO_OF_MEMBERS  TOTAL_OUT_AMT
--------------  -------------  -------------
Kochi                       5            190
Aluva                       4            610
Kakkanad                    4            250
Thrippunithura              2              0

4 rows selected.
```

8. Display staff name, role, shift and salary for staff earning from 32,000 to 56,000 inclusive, excluding GENERAL shift staff. Sort from highest to lowest salary.

CODE:

```
SELECT full_name, role_name, shift_name, salary
FROM staff
WHERE salary BETWEEN 32000 AND 56000
  AND shift_name != 'GENERAL'
ORDER BY salary DESC;
```

OUTPUT:

```
FULL_NAME    ROLE_NAME            SHIFT_NAME  SALARY
-----------  -------------------  ----------  ------
Arun Das     Assistant Librarian  MORNING      48000
Manu Suresh  Library Assistant    MORNING      34000
Rahul Dev    Library Assistant    EVENING      32000

3 rows selected.
```

9. For each shift in the STAFF table, display the number of employees, minimum salary, maximum salary and average salary, rounded to two decimal places.

CODE:

```
SELECT shift_name, COUNT(*) AS no_of_employees,
       MIN(salary) AS min_salary, MAX(salary) AS max_salary,
       ROUND(AVG(salary), 2) AS avg_salary
FROM staff
GROUP BY shift_name
ORDER BY shift_name;
```

OUTPUT:

```
SHIFT_NAME  NO_OF_EMPLOYEES  MIN_SALARY  MAX_SALARY  AVG_SALARY
----------  ---------------  ----------  ----------  ----------
EVENING                   2       30000       32000       31000
GENERAL                   4       52000       72000       58750
MORNING                   2       34000       48000       41000

3 rows selected.
```

10. Display staff members hired before 1 January 2020 whose salary is above 50,000. Show staff ID, name, hire date and salary.

CODE:

```
SELECT staff_id, full_name, hire_date, salary
FROM staff
WHERE hire_date < TO_DATE('01-01-2020', 'DD-MM-YYYY')
  AND salary > 50000
ORDER BY staff_id;
```

OUTPUT:

```
STAFF_ID  FULL_NAME      HIRE_DATE  SALARY
--------  -------------  ---------  ------
     501  Lakshmi Menon  01-JUN-10   72000
     503  Jency Paul     20-JAN-16   55000
     505  Priya Raman    11-SEP-17   56000
     507  Elsa John      13-MAY-19   52000

4 rows selected.
```

11. Display each publisher's name, country and age in years in 2026 (2026 - established_year). Show the oldest publishers first.

CODE:

```
SELECT publisher_name, country,
       (EXTRACT(YEAR FROM SYSDATE) - established_year) AS age
FROM publisher
ORDER BY age DESC;
```

OUTPUT:

```
PUBLISHER_NAME           COUNTRY         AGE
-----------------------  --------------  ---
Oxford University Press  United Kingdom  440
Springer                 Germany         184
Pearson Education        United Kingdom  182
MIT Press                United States    64
DC Books                 India            52
O'Reilly Media           United States    48
HarperCollins            United States    37
Penguin Random House     United States    13

8 rows selected.
```

12. Display countries that have at least two publishers in the database. Show country and number of publishers.

CODE:

```
SELECT country, COUNT(*) AS no_of_publishers
FROM publisher
GROUP BY country
HAVING COUNT(publisher_id) >= 2
ORDER BY country;
```

OUTPUT:

```
COUNTRY         NO_OF_PUBLISHERS
--------------  ----------------
United Kingdom                 2
United States                  4

2 rows selected.
```

13. Display authors born before 1960 who have received at least 2 awards. Show author name, nationality, birth year and awards count, ordered by awards count descending and then birth year ascending.

CODE:

```
SELECT author_name, nationality, birth_year, awards_count
FROM author
WHERE birth_year < 1960
  AND awards_count >= 2
ORDER BY awards_count DESC, birth_year;
```

OUTPUT:

```
AUTHOR_NAME           NATIONALITY     BIRTH_YEAR  AWARDS_COUNT
--------------------  --------------  ----------  ------------
A. P. J. Abdul Kalam  Indian                1931             8
Ronald L. Rivest      American              1947             5
George Orwell         British               1903             3
Andrew S. Tanenbaum   Dutch-American        1944             3
Jane Austen           British               1775             2
Charles E. Leiserson  American              1953             2
Thomas H. Cormen      American              1956             2

7 rows selected.
```

14. For each nationality in AUTHOR, display the number of authors and the total number of awards received. Include only nationalities whose total awards count is at least 5.

CODE:

```
SELECT nationality, COUNT(*) AS no_of_authors,
       SUM(awards_count) AS total_awards
FROM author
GROUP BY nationality
HAVING SUM(awards_count) >= 5
ORDER BY nationality;
```

OUTPUT:

```
NATIONALITY  NO_OF_AUTHORS  TOTAL_AWARDS
-----------  -------------  ------------
American                 5            11
British                  2             5
Indian                   3            21

3 rows selected.
```

15. Display book ID, title, category, publication year and price for books published from 2014 onward with price above 700. Sort by price descending.

CODE:

```
SELECT book_id, title, category, publication_year, price
FROM book
WHERE publication_year >= 2014
  AND price > 700
ORDER BY price DESC;
```

OUTPUT:

```
BOOK_ID  TITLE                                    CATEGORY          PUBLICATION_YEAR  PRICE
-------  ---------------------------------------  ----------------  ----------------  -----
   4013  Artificial Intelligence: A Modern Approach  Computer Science             2021   1199
   4007  Introduction to Algorithms                Computer Science              2022   1099
   4016  Deep Learning                             Computer Science              2016   1050
   4011  Designing Data-Intensive Applications     Computer Science              2017    999
   4008  Modern Operating Systems                  Computer Science              2014    950
   4019  Machine Learning with Python              Computer Science              2022    875
   4012  Learning SQL                              Computer Science              2020    799

7 rows selected.
```

16. Display books whose title contains either the word 'Data' or the word 'Learning', ignoring case. Show title, category, year, pages and rating.

CODE:

```
SELECT title, category, publication_year, pages, rating
FROM book
WHERE LOWER(title) LIKE '%data%'
   OR LOWER(title) LIKE '%learning%'
ORDER BY book_id;
```

OUTPUT:

```
TITLE                                CATEGORY          PUBLICATION_YEAR  PAGES  RATING
-----------------------------------  ----------------  ----------------  -----  ------
Designing Data-Intensive Applications  Computer Science             2017    616     4.8
Learning SQL                         Computer Science              2020    384     4.4
Deep Learning                        Computer Science              2016    800     4.5
Machine Learning with Python         Computer Science              2022    540     4.2

4 rows selected.
```

17. Display books with more than 500 pages and a rating of at least 4.5. Add a computed column named PRICE_PER_100_PAGES showing the approximate price per 100 pages, rounded to two decimals.

CODE:

```
SELECT title, pages, rating,
       ROUND((price / pages) * 100, 2) AS price_per_100_pages
FROM book
WHERE pages > 500
  AND rating >= 4.5
ORDER BY pages DESC;
```

OUTPUT:

```
TITLE                                      PAGES  RATING  PRICE_PER_100_PAGES
----------------------------------------  -----  ------  -------------------
Introduction to Algorithms                 1312     4.7                83.77
Artificial Intelligence: A Modern Approach 1168     4.6               102.65
Modern Operating Systems                   1136     4.5                83.63
Deep Learning                               800     4.5               131.25
Designing Data-Intensive Applications       616     4.8               162.18
Aarachar                                    552     4.6                95.11

6 rows selected.
```

18. Display the category-wise number of books, average price and average rating. Show only categories containing at least 2 books. Round averages to two decimals.

CODE:

```
SELECT category, COUNT(*) AS no_of_books,
       ROUND(AVG(price), 2) AS avg_price,
       ROUND(AVG(rating), 2) AS avg_rating
FROM book
GROUP BY category
HAVING COUNT(book_id) >= 2
ORDER BY category;
```

OUTPUT:

```
CATEGORY          NO_OF_BOOKS  AVG_PRICE  AVG_RATING
----------------  -----------  ---------  ----------
Computer Science            8     983.75        4.53
Fiction                     8     415.75        4.48

2 rows selected.
```

19. Display publication years in which at least 2 books in the BOOK table were published. Show the year, number of books and average price.

CODE:

```
SELECT publication_year, COUNT(*) AS no_of_books,
       ROUND(AVG(price), 2) AS avg_price
FROM book
GROUP BY publication_year
HAVING COUNT(book_id) >= 2
ORDER BY publication_year;
```

OUTPUT:

```
PUBLICATION_YEAR  NO_OF_BOOKS  AVG_PRICE
----------------  -----------  ---------
            1988            2        362
            2014            2      824.5
            2020            2      539.5
            2022            2        987

4 rows selected.
```

20. Categorise every book using a CASE expression: price below 400 is BUDGET, price from 400 to 799 is STANDARD and price 800 or above is PREMIUM. Display book title, price and the computed price category, sorted by price descending.

CODE:

```
SELECT title, price,
       CAST(CASE WHEN price < 400 THEN 'BUDGET'
                 WHEN price BETWEEN 400 AND 799 THEN 'STANDARD'
                 WHEN price >= 800 THEN 'PREMIUM'
            END AS VARCHAR2(14)) AS price_category
FROM book
ORDER BY price DESC;
```

OUTPUT:

```
TITLE                                     PRICE  PRICE_CATEGORY
----------------------------------------  -----  --------------
Artificial Intelligence: A Modern Approach   1199  PREMIUM
Introduction to Algorithms                    1099  PREMIUM
Deep Learning                                 1050  PREMIUM
Designing Data-Intensive Applications          999  PREMIUM
Modern Operating Systems                       950  PREMIUM
Clean Code                                     899  PREMIUM
Machine Learning with Python                   875  PREMIUM
Learning SQL                                   799  STANDARD
Sapiens                                        699  STANDARD
Aarachar                                       525  STANDARD
To Kill a Mockingbird                          499  STANDARD
The God of Small Things                        499  STANDARD
Pride and Prejudice                            450  STANDARD
A Brief History of Time                        425  STANDARD
1984                                           399  BUDGET
The Picture of Dorian Gray                     375  BUDGET
Wings of Fire                                  350  BUDGET
India 2020                                     325  BUDGET
The Alchemist                                  299  BUDGET
Qabar                                          280  BUDGET

20 rows selected.
```

4.2 Joins

1. Display each book title along with its publisher name.

CODE:

```
SELECT b.title, p.publisher_name
FROM book b
JOIN publisher p ON p.publisher_id = b.publisher_id
ORDER BY b.book_id;
```

OUTPUT:

```
TITLE                                      PUBLISHER_NAME
-----------------------------------------  ----------------------
1984                                       Penguin Random House
Pride and Prejudice                        Penguin Random House
Clean Code                                 Pearson Education
Sapiens                                    HarperCollins
Wings of Fire                              DC Books
The God of Small Things                    Penguin Random House
Introduction to Algorithms                 MIT Press
Modern Operating Systems                   Pearson Education
Aarachar                                   DC Books
The Picture of Dorian Gray                 Oxford University Press
Designing Data-Intensive Applications      O'Reilly Media
Learning SQL                               O'Reilly Media
Artificial Intelligence: A Modern Approach Pearson Education
To Kill a Mockingbird                      HarperCollins
India 2020                                 HarperCollins
Deep Learning                              MIT Press
A Brief History of Time                    Oxford University Press
The Alchemist                              HarperCollins
Machine Learning with Python               Springer
Qabar                                      DC Books

20 rows selected.
```

2. Display each library member's name, member type, and the name of the branch to which the member belongs.

CODE:

```
SELECT m.full_name, m.member_type, br.branch_name
FROM member m
JOIN library_branch br ON br.branch_id = m.branch_id
ORDER BY m.member_id;
```

OUTPUT:

```
FULL_NAME        MEMBER_TYPE  BRANCH_NAME
---------------  -----------  -----------------
Aarav Menon      STUDENT      Central Library
Diya Nair        STUDENT      Riverside Branch
Meera Thomas     FACULTY      Knowledge Hub
Rohan Joseph     PUBLIC       Central Library
Ananya Pillai    RESEARCHER   Community Library
Nikhil Raj       STUDENT      Riverside Branch
Sara Mathew      FACULTY      Central Library
Vivek Krishnan   PUBLIC       Knowledge Hub
Ishita Rao       STUDENT      Central Library
Adil Khan        RESEARCHER   Knowledge Hub
Liya George      STUDENT      Community Library
Harish Kumar     PUBLIC       Riverside Branch
Neha Varma       FACULTY      Knowledge Hub
Joel Antony      STUDENT      Central Library
Fathima Noor     RESEARCHER   Riverside Branch

15 rows selected.
```

3. Display the staff name, role, and branch name for all staff members.

CODE:

```
SELECT s.full_name, s.role_name, br.branch_name
FROM staff s
JOIN library_branch br ON br.branch_id = s.branch_id
ORDER BY s.staff_id;
```

OUTPUT:

```
FULL_NAME      ROLE_NAME            BRANCH_NAME
-------------  -------------------  -----------------
Lakshmi Menon  Chief Librarian      Central Library
Arun Das       Assistant Librarian  Central Library
Jency Paul     Librarian            Riverside Branch
Rahul Dev      Library Assistant    Riverside Branch
Priya Raman    Librarian            Knowledge Hub
Manu Suresh    Library Assistant    Knowledge Hub
Elsa John      Librarian            Community Library
Akhil Babu     Library Assistant    Community Library

8 rows selected.
```

4. Display each book title and the names of its authors.

CODE:

```
SELECT b.title, a.author_name
FROM book b
JOIN book_author ba ON ba.book_id = b.book_id
JOIN author a ON a.author_id = ba.author_id
ORDER BY b.book_id, ba.author_order;
```

OUTPUT:

```
TITLE                       AUTHOR_NAME
--------------------------  --------------------
1984                        George Orwell
Pride and Prejudice         Jane Austen
Clean Code                  Robert C. Martin
Sapiens                     Yuval Noah Harari
Wings of Fire               A. P. J. Abdul Kalam
The God of Small Things     Arundhati Roy
Introduction to Algorithms  Thomas H. Cormen
Introduction to Algorithms  Charles E. Leiserson
Introduction to Algorithms  Ronald L. Rivest
Introduction to Algorithms  Clifford Stein
Modern Operating Systems    Andrew S. Tanenbaum
Aarachar                    K. R. Meera
India 2020                  A. P. J. Abdul Kalam

13 rows selected.
```

5. Display the reservation ID, member name, book title, and reservation status for all reservations.

CODE:

```
SELECT r.reservation_id, m.full_name, b.title, r.reservation_status
FROM reservation r
JOIN member m ON m.member_id = r.member_id
JOIN book b ON b.book_id = r.book_id
ORDER BY r.reservation_id;
```

OUTPUT:

```
RESERVATION_ID  FULL_NAME      TITLE                                      RESERVATION_STA
--------------  -------------  -----------------------------------------  ---------------
          8001  Meera Thomas   Introduction to Algorithms                 WAITING
          8002  Ishita Rao     Sapiens                                    WAITING
          8003  Liya George    Designing Data-Intensive Applications      WAITING
          8004  Ananya Pillai  Clean Code                                 FULFILLED
          8005  Diya Nair      Artificial Intelligence: A Modern Approach CANCELLED
          8006  Fathima Noor   Aarachar                                   EXPIRED
          8007  Sara Mathew    Deep Learning                              WAITING
          8008  Adil Khan      Machine Learning with Python               WAITING

8 rows selected.
```

6. Display the fine ID, member name, loan ID, fine amount, and payment status for fines recorded in the library.

CODE:

```
SELECT f.fine_id, m.full_name, f.loan_id, f.amount, f.payment_status
FROM fine f
JOIN member m ON m.member_id = f.member_id
ORDER BY f.fine_id;
```

OUTPUT:

```
FINE_ID  FULL_NAME       LOAN_ID  AMOUNT  PAYMENT_STATUS
-------  --------------  -------  ------  --------------
   9001  Aarav Menon        7002      90  UNPAID
   9002  Adil Khan          7004     250  PARTIAL
   9003  Sara Mathew        7007      30  PAID
   9004  Neha Varma         7010      70  PAID
   9005  Nikhil Raj         7011      60  WAIVED
   9006  Vivek Krishnan     7013      70  PAID
   9007  Joel Antony        7015     100  PARTIAL
   9008  Harish Kumar       7016     480  UNPAID

8 rows selected.
```

7. Display all library branches and the names of members registered at each branch. Include branches that have no members.

CODE:

```
SELECT br.branch_name, m.full_name
FROM library_branch br
LEFT JOIN member m ON m.branch_id = br.branch_id
ORDER BY br.branch_id, m.member_id;
```

OUTPUT:

```
BRANCH_NAME        FULL_NAME
-----------------  --------------
Central Library    Aarav Menon
Central Library    Rohan Joseph
Central Library    Sara Mathew
Central Library    Ishita Rao
Central Library    Joel Antony
Riverside Branch   Diya Nair
Riverside Branch   Nikhil Raj
Riverside Branch   Harish Kumar
Riverside Branch   Fathima Noor
Knowledge Hub      Meera Thomas
Knowledge Hub      Vivek Krishnan
Knowledge Hub      Adil Khan
Knowledge Hub      Neha Varma
Community Library  Ananya Pillai
Community Library  Liya George

15 rows selected.
```

8. Display all books and their physical copy IDs. Include books that do not have any physical copies.

CODE:

```
SELECT b.title, c.copy_id
FROM book b
LEFT JOIN book_copy c ON c.book_id = b.book_id
ORDER BY b.book_id, c.copy_id;
```

OUTPUT:

```
TITLE                                     COPY_ID
----------------------------------------  -------
1984                                         6001
1984                                         6002
Pride and Prejudice                          6003
Clean Code                                   6004
Clean Code                                   6005
Sapiens                                      6006
Wings of Fire                                6007
The God of Small Things                      6008
Introduction to Algorithms                   6009
Introduction to Algorithms                   6010
Modern Operating Systems                     6011
Aarachar                                     6012
The Picture of Dorian Gray                   6013
Designing Data-Intensive Applications        6014
Designing Data-Intensive Applications        6024
Learning SQL                                 6015
Artificial Intelligence: A Modern Approach   6016
To Kill a Mockingbird                        6017
India 2020                                   6018
Deep Learning                                6019
A Brief History of Time                      6020
The Alchemist                                6021
Machine Learning with Python                 6022
Qabar                                        6023

24 rows selected.
```

9. Display all members and their loan IDs. Include members who have never borrowed a book.

CODE:

```
SELECT m.full_name, l.loan_id
FROM member m
LEFT JOIN loan l ON l.member_id = m.member_id
ORDER BY m.member_id, l.loan_id;
```

OUTPUT:

```
FULL_NAME        LOAN_ID
---------------  -------
Aarav Menon         7002
Aarav Menon         7014
Diya Nair           7001
Meera Thomas        7006
Rohan Joseph        7005
Ananya Pillai       7003
Nikhil Raj          7011
Sara Mathew         7007
Vivek Krishnan      7013
Ishita Rao          7008
Adil Khan           7004
Liya George         7009
Harish Kumar        7016
Neha Varma          7010
Joel Antony         7015
Fathima Noor        7012

16 rows selected.
```

10. Display all authors and the titles of books associated with them through BOOK_AUTHOR. Include authors who are not linked to any book.

CODE:

```
SELECT a.author_name, b.title
FROM author a
LEFT JOIN book_author ba ON ba.author_id = a.author_id
LEFT JOIN book b ON b.book_id = ba.book_id
ORDER BY a.author_id, b.title;
```

OUTPUT:

```
AUTHOR_NAME           TITLE
--------------------  --------------------------
George Orwell         1984
Jane Austen           Pride and Prejudice
Robert C. Martin      Clean Code
Yuval Noah Harari     Sapiens
A. P. J. Abdul Kalam  India 2020
A. P. J. Abdul Kalam  Wings of Fire
Arundhati Roy         The God of Small Things
Thomas H. Cormen      Introduction to Algorithms
Charles E. Leiserson  Introduction to Algorithms
Ronald L. Rivest      Introduction to Algorithms
Clifford Stein        Introduction to Algorithms
Andrew S. Tanenbaum   Modern Operating Systems
K. R. Meera           Aarachar

13 rows selected.
```

11. Display all publishers and the titles of books published by them. Include publishers that have not published any books.

CODE:

```
SELECT p.publisher_name, b.title
FROM publisher p
LEFT JOIN book b ON b.publisher_id = p.publisher_id
ORDER BY p.publisher_id, b.book_id;
```

OUTPUT:

```
PUBLISHER_NAME           TITLE
-----------------------  ----------------------------------------
Pearson Education        Clean Code
Pearson Education        Modern Operating Systems
Pearson Education        Artificial Intelligence: A Modern Approach
Penguin Random House     1984
Penguin Random House     Pride and Prejudice
Penguin Random House     The God of Small Things
O'Reilly Media           Designing Data-Intensive Applications
O'Reilly Media           Learning SQL
HarperCollins            Sapiens
HarperCollins            To Kill a Mockingbird
HarperCollins            India 2020
HarperCollins            The Alchemist
Oxford University Press  The Picture of Dorian Gray
Oxford University Press  A Brief History of Time
MIT Press                Introduction to Algorithms
MIT Press                Deep Learning
DC Books                 Wings of Fire
DC Books                 Aarachar
DC Books                 Qabar
Springer                 Machine Learning with Python

20 rows selected.
```

12. Display branch name, city, and staff name using a RIGHT OUTER JOIN. Include all branches, even if a branch has no staff.

CODE:

```
SELECT br.branch_name, br.city, s.full_name
FROM staff s
RIGHT JOIN library_branch br ON br.branch_id = s.branch_id
ORDER BY br.branch_id, s.staff_id;
```

OUTPUT:

```
BRANCH_NAME        CITY             FULL_NAME
-----------------  ---------------  -------------
Central Library    Kochi            Lakshmi Menon
Central Library    Kochi            Arun Das
Riverside Branch   Aluva            Jency Paul
Riverside Branch   Aluva            Rahul Dev
Knowledge Hub      Kakkanad         Priya Raman
Knowledge Hub      Kakkanad         Manu Suresh
Community Library  Thrippunithura   Elsa John
Community Library  Thrippunithura   Akhil Babu

8 rows selected.
```

13. Display book title, copy ID, and availability status using a RIGHT OUTER JOIN. Include all physical copies in BOOK_COPY.

CODE:

```
SELECT b.title, c.copy_id, c.availability_status
FROM book b
RIGHT JOIN book_copy c ON c.book_id = b.book_id
ORDER BY c.copy_id;
```

OUTPUT:

```
TITLE                                      COPY_ID  AVAILABILITY_STATUS
-----------------------------------------  -------  ------------------
1984                                          6001  AVAILABLE
1984                                          6002  ISSUED
Pride and Prejudice                           6003  AVAILABLE
Clean Code                                    6004  ISSUED
Clean Code                                    6005  AVAILABLE
Sapiens                                       6006  RESERVED
Wings of Fire                                 6007  AVAILABLE
The God of Small Things                       6008  AVAILABLE
Introduction to Algorithms                    6009  ISSUED
Introduction to Algorithms                    6010  AVAILABLE
Modern Operating Systems                      6011  AVAILABLE
Aarachar                                      6012  AVAILABLE
The Picture of Dorian Gray                    6013  REPAIR
Designing Data-Intensive Applications         6014  ISSUED
Learning SQL                                  6015  AVAILABLE
Artificial Intelligence: A Modern Approach    6016  AVAILABLE
To Kill a Mockingbird                         6017  ISSUED
India 2020                                    6018  AVAILABLE
Deep Learning                                 6019  AVAILABLE
A Brief History of Time                       6020  AVAILABLE
The Alchemist                                 6021  LOST
Machine Learning with Python                  6022  AVAILABLE
Qabar                                         6023  AVAILABLE
Designing Data-Intensive Applications         6024  RESERVED

24 rows selected.
```

14. Display publisher name, book title, and price using a RIGHT OUTER JOIN. Include all books, even if a matching publisher record is not found.

CODE:

```
SELECT p.publisher_name, b.title, b.price
FROM publisher p
RIGHT JOIN book b ON b.publisher_id = p.publisher_id
ORDER BY b.book_id;
```

OUTPUT:

```
PUBLISHER_NAME           TITLE                                      PRICE
-----------------------  -----------------------------------------  -----
Penguin Random House     1984                                         399
Penguin Random House     Pride and Prejudice                          450
Pearson Education        Clean Code                                   899
HarperCollins            Sapiens                                      699
DC Books                 Wings of Fire                                350
Penguin Random House     The God of Small Things                      499
MIT Press                Introduction to Algorithms                  1099
Pearson Education        Modern Operating Systems                     950
DC Books                 Aarachar                                     525
Oxford University Press  The Picture of Dorian Gray                   375
O'Reilly Media           Designing Data-Intensive Applications        999
O'Reilly Media           Learning SQL                                 799
Pearson Education        Artificial Intelligence: A Modern Approach  1199
HarperCollins            To Kill a Mockingbird                        499
HarperCollins            India 2020                                   325
MIT Press                Deep Learning                               1050
Oxford University Press  A Brief History of Time                     425
HarperCollins            The Alchemist                                299
Springer                 Machine Learning with Python                 875
DC Books                 Qabar                                        280

20 rows selected.
```

15. Display all library branches and all members, showing matching branch-member records, branches without members, and members without a matching branch.

CODE:

```
SELECT br.branch_name, m.full_name
FROM library_branch br
FULL OUTER JOIN member m ON m.branch_id = br.branch_id
ORDER BY br.branch_id, m.member_id;
```

OUTPUT:

```
BRANCH_NAME         FULL_NAME
------------------  --------------
Central Library     Aarav Menon
Central Library     Rohan Joseph
Central Library     Sara Mathew
Central Library     Ishita Rao
Central Library     Joel Antony
Riverside Branch    Diya Nair
Riverside Branch    Nikhil Raj
Riverside Branch    Harish Kumar
Riverside Branch    Fathima Noor
Knowledge Hub       Meera Thomas
Knowledge Hub       Vivek Krishnan
Knowledge Hub       Adil Khan
Knowledge Hub       Neha Varma
Community Library   Ananya Pillai
Community Library   Liya George

15 rows selected.
```

16. Display all books and all reservations, including books that have never been reserved and reservation records without a matching book.

CODE:

```
SELECT b.title, r.reservation_id
FROM book b
FULL OUTER JOIN reservation r ON r.book_id = b.book_id
ORDER BY b.book_id, r.reservation_id;
```

OUTPUT:

```
TITLE                                      RESERVATION_ID
-----------------------------------------  --------------
1984
Pride and Prejudice
Clean Code                                           8004
Sapiens                                              8002
Wings of Fire
The God of Small Things
Introduction to Algorithms                           8001
Modern Operating Systems
Aarachar                                             8006
The Picture of Dorian Gray
Designing Data-Intensive Applications                8003
Learning SQL
Artificial Intelligence: A Modern Approach           8005
To Kill a Mockingbird
India 2020
Deep Learning                                        8007
A Brief History of Time
The Alchemist
Machine Learning with Python                         8008
Qabar

20 rows selected.
```

17. Display all members and all fine records, including members without fines and fine records without a matching member.

CODE:

```
SELECT m.full_name, f.fine_id
FROM member m
FULL OUTER JOIN fine f ON f.member_id = m.member_id
ORDER BY m.member_id, f.fine_id;
```

OUTPUT:

```
FULL_NAME       FINE_ID
--------------  -------
Aarav Menon        9001
Diya Nair
Meera Thomas
Rohan Joseph
Ananya Pillai
Nikhil Raj         9005
Sara Mathew        9003
Vivek Krishnan     9006
Ishita Rao
Adil Khan          9002
Liya George
Harish Kumar       9008
Neha Varma         9004
Joel Antony        9007
Fathima Noor

15 rows selected.
```

18. Display book title, copy ID, and branch name using a NATURAL JOIN between BOOK_COPY and LIBRARY_BRANCH.

CODE:

```
SELECT b.title, c.copy_id, l.branch_name
FROM book_copy c
NATURAL JOIN library_branch l
JOIN book b ON b.book_id = c.book_id
ORDER BY c.copy_id;
```

OUTPUT:

```
TITLE                                     COPY_ID  BRANCH_NAME
----------------------------------------  -------  -----------------
1984                                         6001  Central Library
1984                                         6002  Riverside Branch
Pride and Prejudice                          6003  Central Library
Clean Code                                   6004  Central Library
Clean Code                                   6005  Knowledge Hub
Sapiens                                      6006  Community Library
Wings of Fire                                6007  Riverside Branch
The God of Small Things                      6008  Central Library
Introduction to Algorithms                   6009  Knowledge Hub
Introduction to Algorithms                   6010  Central Library
Modern Operating Systems                     6011  Community Library
Aarachar                                     6012  Community Library
The Picture of Dorian Gray                   6013  Riverside Branch
Designing Data-Intensive Applications        6014  Knowledge Hub
Learning SQL                                 6015  Central Library
Artificial Intelligence: A Modern Approach   6016  Knowledge Hub
To Kill a Mockingbird                        6017  Riverside Branch
India 2020                                   6018  Central Library
Deep Learning                                6019  Knowledge Hub
A Brief History of Time                      6020  Community Library
The Alchemist                                6021  Central Library
Machine Learning with Python                 6022  Riverside Branch
Qabar                                        6023  Community Library
Designing Data-Intensive Applications        6024  Central Library

24 rows selected.
```

19. Display member name, book title, copy accession number, issue date, and loan status for books that have been loaned.

CODE:

```
SELECT m.full_name, b.title, c.accession_no, l.issue_date, l.loan_status
FROM loan l
JOIN member m ON m.member_id = l.member_id
JOIN book_copy c ON c.copy_id = l.copy_id
JOIN book b ON b.book_id = c.book_id
ORDER BY l.loan_id;
```

OUTPUT:

```
FULL_NAME       TITLE                                ACCESSION_NO  ISSUE_DAT  LOAN_STATUS
--------------  -----------------------------------  ------------  ---------  -----------
Diya Nair       1984                                 ACC-R-0001    20-AUG-26  ISSUED
Aarav Menon     Clean Code                           ACC-C-0003    12-AUG-26  OVERDUE
Ananya Pillai   Introduction to Algorithms           ACC-K-0002    24-AUG-26  ISSUED
Adil Khan       Designing Data-Intensive Applications ACC-K-0003   01-AUG-26  OVERDUE
Rohan Joseph    To Kill a Mockingbird                ACC-R-0004    22-AUG-26  ISSUED
Meera Thomas    1984                                 ACC-C-0001    02-JUL-26  RETURNED
Sara Mathew     Pride and Prejudice                  ACC-C-0002    11-JUN-26  RETURNED
Ishita Rao      Wings of Fire                        ACC-R-0002    18-JUL-26  RETURNED
Liya George     The God of Small Things              ACC-C-0004    21-JUL-26  RETURNED
Neha Varma      Introduction to Algorithms           ACC-C-0005    02-MAY-26  RETURNED
Nikhil Raj      Modern Operating Systems             ACC-T-0002    14-APR-26  RETURNED
Fathima Noor    Aarachar                             ACC-T-0003    05-MAR-26  RETURNED
Vivek Krishnan  The Picture of Dorian Gray           ACC-R-0003    01-FEB-26  RETURNED
Aarav Menon     Learning SQL                         ACC-C-0006    12-JAN-26  RETURNED
Joel Antony     India 2020                           ACC-C-0007    10-DEC-25  RETURNED
Harish Kumar    The Alchemist                        ACC-C-0008    01-JUN-26  LOST

16 rows selected.
```

20. Display book title, publisher name, author name, and contribution role for books linked to authors.

CODE:

```
SELECT b.title, p.publisher_name, a.author_name,
       ba.contribution_role
FROM book b
JOIN publisher p ON p.publisher_id = b.publisher_id
JOIN book_author ba ON ba.book_id = b.book_id
JOIN author a ON a.author_id = ba.author_id
ORDER BY b.book_id, ba.author_order;
```

OUTPUT:

```
TITLE                      PUBLISHER_NAME        AUTHOR_NAME           CONTRIBUTION_ROLE
-------------------------  --------------------  --------------------  -----------------
1984                       Penguin Random House  George Orwell         AUTHOR
Pride and Prejudice        Penguin Random House  Jane Austen           AUTHOR
Clean Code                 Pearson Education     Robert C. Martin      AUTHOR
Sapiens                    HarperCollins         Yuval Noah Harari     AUTHOR
Wings of Fire              DC Books              A. P. J. Abdul Kalam  AUTHOR
The God of Small Things    Penguin Random House  Arundhati Roy         AUTHOR
Introduction to Algorithms MIT Press             Thomas H. Cormen      AUTHOR
Introduction to Algorithms MIT Press             Charles E. Leiserson  CO-AUTHOR
Introduction to Algorithms MIT Press             Ronald L. Rivest      CO-AUTHOR
Introduction to Algorithms MIT Press             Clifford Stein        CO-AUTHOR
Modern Operating Systems   Pearson Education     Andrew S. Tanenbaum   AUTHOR
Aarachar                   DC Books              K. R. Meera           AUTHOR
India 2020                 HarperCollins         A. P. J. Abdul Kalam  AUTHOR

13 rows selected.
```

4.3 Views

1. Create a view named BOOK_DETAILS to display the book ID, book title, publisher ID, and price from the BOOK table.

CODE:

```
CREATE VIEW book_details AS
SELECT book_id, title, publisher_id, price
FROM book;
```

OUTPUT:

```
View created.
```

2. Create a view named MEMBER_DETAILS to display the member ID, member name, member type, and status of all library members.

CODE:

```
CREATE VIEW member_details AS
SELECT member_id, full_name, member_type, status
FROM member;
```

OUTPUT:

```
View created.
```

3. Create a view named MEMBER_BRANCH_DETAILS to display member name, member type, branch name, and city by joining MEMBER and LIBRARY_BRANCH.

CODE:

```
CREATE VIEW member_branch_details AS
SELECT m.full_name, m.member_type, br.branch_name, br.city
FROM member m
JOIN library_branch br ON br.branch_id = m.branch_id;
```

OUTPUT:

```
View created.
```

4. Create a view named BOOK_PUBLISHER_DETAILS to display book title, publisher name, and price by joining BOOK and PUBLISHER.

CODE:

```
CREATE VIEW book_publisher_details AS
SELECT b.title, p.publisher_name, b.price
FROM book b
JOIN publisher p ON p.publisher_id = b.publisher_id;
```

OUTPUT:

```
View created.
```

5. Create a view named LOAN_DETAILS to display member name, book title, issue date, due date, and loan status by joining the appropriate tables.

CODE:

```
CREATE VIEW loan_details AS
SELECT m.full_name, b.title, l.issue_date, l.due_date, l.loan_status
FROM loan l
JOIN member m ON m.member_id = l.member_id
JOIN book_copy c ON c.copy_id = l.copy_id
JOIN book b ON b.book_id = c.book_id;
```

OUTPUT:

```
View created.
```

6. Display the names and branch names of all members using the MEMBER_BRANCH_DETAILS view.

CODE:

```
SELECT full_name, branch_name
FROM member_branch_details
ORDER BY full_name;
```

OUTPUT:

```
FULL_NAME       BRANCH_NAME
--------------  -----------------
Aarav Menon     Central Library
Adil Khan       Knowledge Hub
Ananya Pillai   Community Library
Diya Nair       Riverside Branch
Fathima Noor    Riverside Branch
Harish Kumar    Riverside Branch
Ishita Rao      Central Library
Joel Antony     Central Library
Liya George     Community Library
Meera Thomas    Knowledge Hub
Neha Varma      Knowledge Hub
Nikhil Raj      Riverside Branch
Rohan Joseph    Central Library
Sara Mathew     Central Library
Vivek Krishnan  Knowledge Hub

15 rows selected.
```

7. Display books from the BOOK_PUBLISHER_DETAILS view whose price is greater than 500.

CODE:

```
SELECT title, publisher_name, price
FROM book_publisher_details
WHERE price > 500
ORDER BY price DESC;
```

OUTPUT:

```
TITLE                                      PUBLISHER_NAME     PRICE
-----------------------------------------  -----------------  -----
Artificial Intelligence: A Modern Approach  Pearson Education   1199
Introduction to Algorithms                  MIT Press           1099
Deep Learning                               MIT Press           1050
Designing Data-Intensive Applications       O'Reilly Media       999
Modern Operating Systems                    Pearson Education    950
Clean Code                                  Pearson Education    899
Machine Learning with Python                Springer             875
Learning SQL                                O'Reilly Media       799
Sapiens                                     HarperCollins        699
Aarachar                                    DC Books             525

10 rows selected.
```

4.4 TCL and DCL Commands

1. Increase the price of all books published by publisher 201 by 10%. Save the changes permanently using COMMIT.

CODE:

```
UPDATE book SET price = price * 1.10
WHERE publisher_id = 201;
COMMIT;
```

OUTPUT:

```
3 rows updated.
Commit complete.
```

2. Update the status of library member 1006 from EXPIRED to ACTIVE and permanently save the change using COMMIT.

CODE:

```
UPDATE member SET status = 'ACTIVE'
WHERE member_id = 1006 AND status = 'EXPIRED';
COMMIT;
```

OUTPUT:

```
1 row updated.
Commit complete.
```

3. Update the availability status of book copy 6001 to REPAIR, so that it can no longer be issued. Commit the transaction.

CODE:

```
UPDATE book_copy SET availability_status = 'REPAIR'
WHERE copy_id = 6001;
COMMIT;
```

OUTPUT:

```
1 row updated.
Commit complete.
```

4. Update the price of book 4001. Create a savepoint named price_update, perform another update on the same book, and then roll back to the price_update savepoint.

CODE:

```
UPDATE book SET price = 500 WHERE book_id = 4001;
SAVEPOINT price_update;
UPDATE book SET price = 900 WHERE book_id = 4001;
ROLLBACK TO price_update;
```

OUTPUT:

```
1 row updated.
Savepoint created.
1 row updated.
Rollback complete.
```

5. Perform the following operations in a single transaction: update the status of a member, create a savepoint named member_update, update the price of a book, roll back to member_update, and commit the transaction.

CODE:

```
UPDATE member SET status = 'SUSPENDED' WHERE member_id = 1007;
SAVEPOINT member_update;
UPDATE book SET price = 1200 WHERE book_id = 4007;
ROLLBACK TO member_update;
COMMIT;
```

OUTPUT:

```
1 row updated.
Savepoint created.
1 row updated.
Rollback complete.
Commit complete.
```

6. Perform two different updates on the BOOK table with a savepoint between them. Roll back only the second update using ROLLBACK TO SAVEPOINT.

CODE:

```
UPDATE book SET price = 420 WHERE book_id = 4002;
SAVEPOINT second_update;
UPDATE book SET price = 780 WHERE book_id = 4005;
ROLLBACK TO second_update;
```

OUTPUT:

```
1 row updated.
Savepoint created.
1 row updated.
Rollback complete.
```

7. Grant SELECT permission on the BOOK table to user LIBUSER.

CODE:

```
GRANT SELECT ON book TO libuser;
```

OUTPUT:

```
Grant succeeded.
```

8. Grant SELECT permission on the BOOK_DETAILS view to user LIBUSER.

CODE:

```
GRANT SELECT ON book_details TO libuser;
```

OUTPUT:

```
Grant succeeded.
```

9. Grant SELECT and INSERT privileges on the MEMBER table to user LIBUSER.

CODE:

```
GRANT SELECT, INSERT ON member TO libuser;
```

OUTPUT:

```
Grant succeeded.
```

10. Grant SELECT, UPDATE, and DELETE privileges on the BOOK_COPY table to user LIBUSER.

CODE:

```
GRANT SELECT, UPDATE, DELETE ON book_copy TO libuser;
```

OUTPUT:

```
Grant succeeded.
```

11. Grant SELECT permission on the MEMBER_BRANCH_DETAILS view to user LIBUSER.

CODE:

```
GRANT SELECT ON member_branch_details TO libuser;
```

OUTPUT:

```
Grant succeeded.
```

12. Revoke the SELECT privilege on the BOOK table from user LIBUSER.

CODE:

```
REVOKE SELECT ON book FROM libuser;
```

OUTPUT:

```
Revoke succeeded.
```

13. Revoke the INSERT privilege on the MEMBER table from user LIBUSER.

CODE:

```
REVOKE INSERT ON member FROM libuser;
```

OUTPUT:

```
Revoke succeeded.
```

14. Revoke the UPDATE and DELETE privileges on the BOOK_COPY table from user LIBUSER.

CODE:

```
REVOKE UPDATE, DELETE ON book_copy FROM libuser;
```

OUTPUT:

```
Revoke succeeded.
```

15. Revoke all previously granted privileges on the BOOK_DETAILS view from user LIBUSER.

CODE:

```
REVOKE ALL ON book_details FROM libuser;
```

OUTPUT:

```
Revoke succeeded.
```